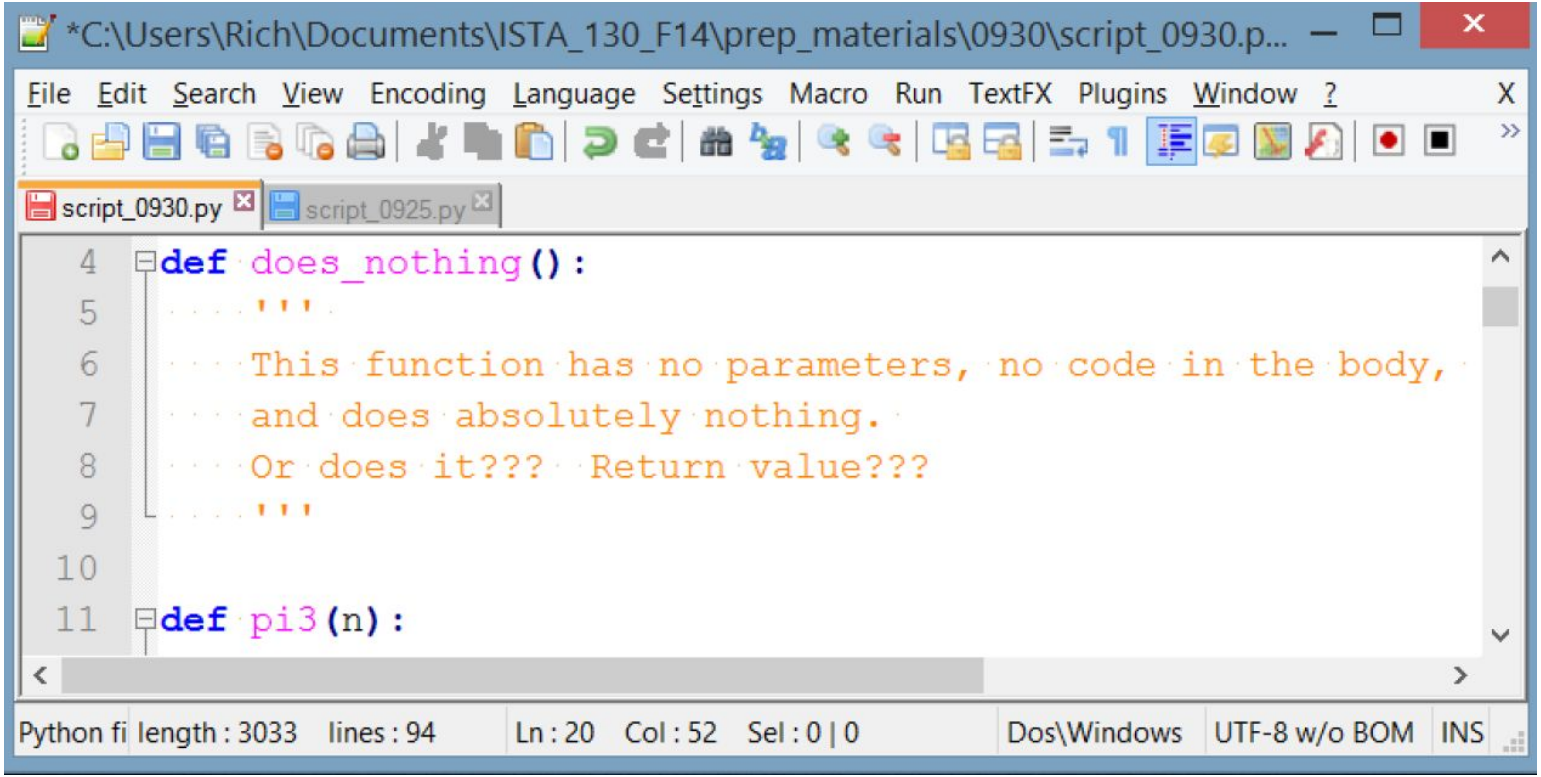
ISTA 130

Conditional Statements II


*C:\Users\Rich\Documents\ISTA_130_F14\prep_materials\0930\script_0930.p...
File Edit Search View Encoding Language Settings Macro Run TextFX Plugins Window ? X
script_0930.py
script_0925.py
4 def does_nothing():
5 '''
6 This function has no parameters, no code in the body,
7 and does absolutely nothing.
8 Or does it??? Return value???
9 '''
10
11 def pi3(n):
Python fi length : 3033 lines : 94 Ln : 20 Col : 52 Sel : 0 | 0 Dos\Windows UTF-8 w/o BOM INS

None Is Not Nothing

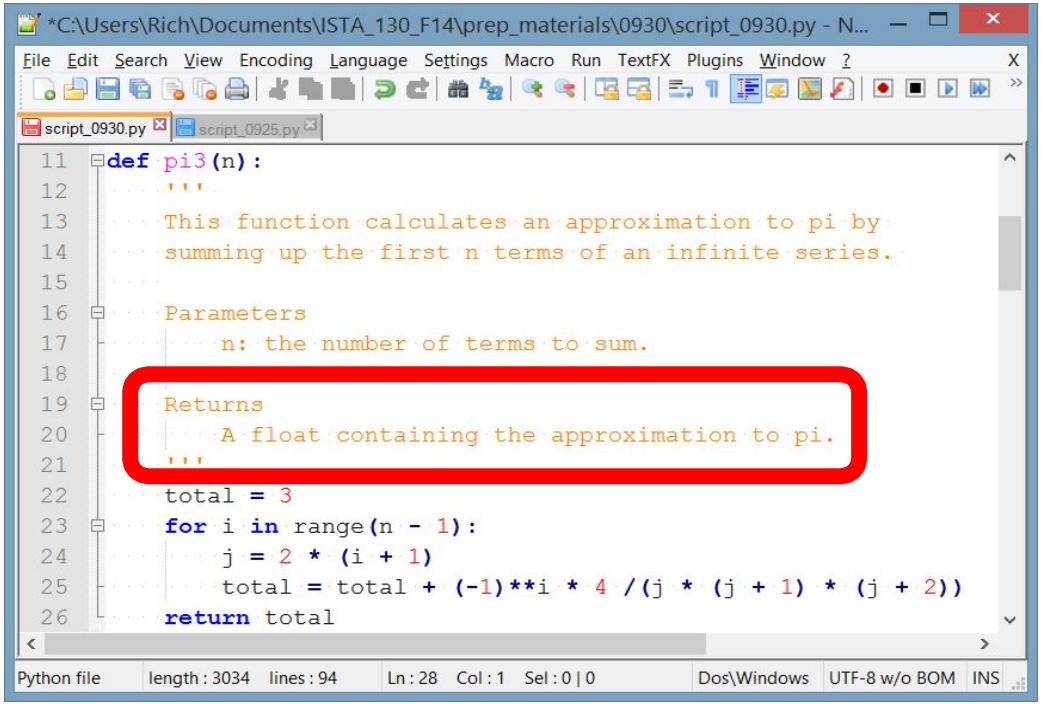
None is an actual object in Python. It is like a turtle or an integer or a string.

If you don't have a return statement in a function, Python still returns None for you.

Let's take a look at does_nothing in the interactive shell...

Documenting return Statements

Describe the type and meaning of the object you return:



HW – TYPECASTING

The only times you need to typecast is when you use the input() function, in string concatenation, and (we'll see later) when you read from a file.

Unnecessary operations degrade program readability and maintainability.

Decisions With Many Options

```
if candy_color == 'red':
    print('Cherry')
elif candy_color == 'green':
    print('Lime')
elif candy_color == 'yellow':
    print('Lemon')
elif candy_color == 'orange':
    print('Orange')
else:
    print('No clue!')
```

- Note: only **one** of these print statements will execute
  - Python enters the first condition that evaluates to True, executes the body of that compound statement, and skips all the rest
    - (just like your roommate did with the grocery list)

Errors In Conditionals

How much sense does this make:

```
if candy_color == 'red':
    print('Cherry')
elif candy_color == 'green':
    print('Lime')
elif candy_color == 'yellow':
    print('Lemon')
else:
    print('No clue!')
elif candy_color == 'orange':
    print('Orange')
```

- Since `else` takes care of "all other cases" there's nothing left for that last `elif`, it's an error

Errors In Conditionals

How about this:

```
if candy_color == 'red':
    print('Cherry')
elif candy_color == 'green':
    print('Lime')
else:
    print('No clue!')
else:
    print('Hello...?')
```

- Again `else` takes care of "all other cases", so that second `else` is an error

Errors In Conditionals

What about this:

```
if candy_color == 'red':
   print('Cherry')
elif candy_color == 'green':
   print('Lime')
else candy_color == 'orange':
   print('Huh?')
```

- The `if` and the `elif` both need conditions, the `else` can't have one, it wouldn't make sense.
  - The `else` statement takes care of *all other cases*.

Logical Operators

To make our *boolean* expressions more interesting we have a set of *logical* operators:

and

or

not

- Used with the *comparison* operators, these allow us to make more complex conditions.
- As with mathematical expressions, you can put parentheses in your boolean expressions for grouping and/or for clarity.

```
if (x > 50) and (x < 100):
    print('x is kinda big!')
```

```
if (x > 100) or (y > 100):
    print('at least one of \'em is big!')
```

```
if not (x > 100):
    print('x is not big')
```

Logical Operators

- ```
  if (age > 12) and (age < 20):
      print('You are a teenager.')
  ```
- ```
  if (ounces == 12) or (ounces == 20):
      print('It\'s some kind of pint')
  ```
- ```
  if not (color == 'blue'):
      print('the turtle is not sad :)')
  ```
- ```
  if (color != 'blue'):
      print('the turtle is not sad :)')
  ```
- ```
  if not not (color != 'blue'):
      print('the turtle is not sad, but I am :(')
  ```
- ```
  if not (color == 'green' and pensize > 5):
      print('the turtle is sad')
      print('he prefers to be big and green')
  ```

Logical Operators ?

- ```
  if (age < 12) and (age > 20):
      print('That\'s really amazing!')
  ```
- ```
  if (age < 12) and (age < 20):
      print('Uh, okay.')
  ```
- ```
  if (age < 20) or (age < 12):
      print('Hmmmmm...')
  ```
  ```

Grading

```
if (grade >= 90.0):
    print('A')
elif (grade < 90.0 and grade >= 80.0):
    print('B')
elif (grade < 80.0 and grade >= 70.0):
    print('C')
elif (grade < 70.0 and grade >= 60.0):
    print('D')
else:
    print('E')
```

- Do you see anything redundant in this code?
  - Why didn't I have to do this: `elif grade < 60.0:`

Grading, improved

```
if grade >= 90.0:
    print('A')
elif grade >= 80.0:
    print('B')
elif grade >= 70.0:
    print('C')
elif grade >= 60.0:
    print('D')
else:
    print('E')
```

- You can think of this as "filtering" (like a sieve)
  - the first condition filters off all of the grades $>= 90.0$
  - only grades that are < 90.0 make it to the next condition
  - ...
  - by the time you get down to the `else`, the only grades left are already guaranteed to be < 60.0
- This code is shorter and more clear.

Selection Statements

Conditionals are sometimes called selection statements because they are about choosing – choosing which code to execute based on some test (the condition).

`if` Statements vs. `for` Loops

if statements are for choosing whether or not to execute code based on a test.

for loops are for repeating code.

`if` Statements vs. `for` Loops

If you want the option to skip a piece of code, use an if statement.

If you want a section of code to execute more than once, use a for loop.

Outline

Conditional Statements
- Boolean Expressions

Example
- Example
- Short Circuit Evaluation

Exercises

`weapons.py`!

Boolean Expressions

- What does this do?

```
if True:
    print('Programming is fun!')
```

- How about this?

```
if False:
    print('I like cupcakes.')
```

- Python evaluates the condition expression and executes the body *only* if the condition evaluates to `True`
  - no matter how complicated the expression is, it still always boils down to either `True` or `False`

Boolean Expressions With Logical Operators

Sometimes reading Boolean expressions can be confusing.

- It might help to read it out loud in this form:

  "Is it the case that *both* ___ **and** ___ are `True`?"

  "Is it the case that *either* ___ **or** ___ is `True`?"

- For example:
  - `X or Y` read as →
    "Is it the case that *either* X or Y is True?"
  - `False or True` read as →
    "Is it the case that *either* False or True is True?"
  - `X and Y` read as →
    "Is it the case that *both* X and Y are True?"
  - `True and False` →
    "Is it the case that *both* True and False are True?"

Truth Tables

X	Y	X and Y
True	True	True
True	False	False
False	True	False
False	False	False

X	Y	X or Y
True	True	True
True	False	True
False	True	True
False	False	False

X	not X
True	False
False	True

Nested Conditionals

- The body of a conditional statement can of course contain other conditional statements
  - We call this *nesting*

```
if condition:
    ...
    if condition:
        ...
    else:
        ...
    ...
elif condition:
    ...
    if condition:
        ...
    ...
else:
    ...
```

Nested Conditionals

- Can you do this?

```
if condition:
    if condition:
        if condition:
            if condition:
                if condition:
                    ...
    ...
else:
    ...
```

- Certainly. You can *nest* as deeply as you need to.
  - A conditional *statement* can contain as many *statements* as you need it to.
    - a compound statement is a statement
- (However, if your code looks like the example above, there's probably a better way to write it.)

Example: Amusement Park Rides

Rides at amusement parks often have a variety of restrictions on who can and cannot ride.

- Let's look at a simple example with:
  - two rides: "Zipper", "Cycle Jump"
  - two age groups: "Adult", "Child"
  - the "Zipper" is restricted to adults
  - the "Cycle Jump" is restricted to children
- We'll write a function that takes the name of the ride and prints the age group that can ride it.

Amusement Park Rides

```
def can_ride(ride):
    if ride == 'Cycle Jump':
        print('Children only')
    elif ride == 'Zipper':
        print('Adults only')
    else:
        print('Unidentified Ride Name')
    return None
# ----------------------------------------
def main():
    can_ride('Ziper') # what will this print?
    can_ride('Zipper') # what will this print?
    can_ride('Cycle Jump') # what will this print?
```

Amusement Park Rides

Let's extend our function to take an `age` parameter:

- the `age` parameter will either be "Adult" or "Child"
- we'll the function `return` either "Yes" or "No" depending on whether the given age group can ride the given ride.

Rides With Age Group, First Attempt

```
1  def can_ride(ride, age):
2      result = '?'
3      if ride == 'Cycle Jump' and age == 'Child':
4          result = 'Yes'
5      elif ride == 'Cycle Jump' and age == 'Adult':
6          result = 'No'
7      elif ride == 'Zipper' and age == 'Adult':
8          result = 'Yes'
9      elif ride == 'Zipper' and age == 'Child':
10         result = 'No'
11     else:
12         result = 'Unidentified Ride Name'
13     return result
```

```
can_ride('Child', 'Zipper')# what will this return?

can_ride('Zipper', 'Child')# what will this return?

can_ride('Zipper', 'Adult')# what will this return?
```

Rides With Age Group, First Attempt

```
1  def can_ride(ride, age):
2      result = '?'
3      if ride == 'Cycle Jump' and age == 'Child':
4          result = 'Yes'
5      elif ride == 'Cycle Jump' and age == 'Adult':
6          result = 'No'
7      elif ride == 'Zipper' and age == 'Adult':
8          result = 'Yes'
9      elif ride == 'Zipper' and age == 'Child':
10         result = 'No'
11     else:
12         result = 'Unidentified Ride Name'
13     return result
```

- Here we've enumerated every possible case (i.e. all combinations of ride and age group)
- Could we write this function another way?

Rides With Age Group, Second Attempt

```
def can_ride(ride, age):
    result = '?'
    if ride == 'Cycle Jump':
        if age == 'Child':
            result = 'Yes'
        else:
            result = 'No'
    elif ride == 'Zipper':
        if age == 'Adult':
            result = 'Yes'
        else:
            result = 'No'
    else:
        result = 'Unidentified Ride Name'
    return result
```

```
can_ride('Child', 'Zipper')# what will this return?
can_ride('Zipper', 'Child')# what will this return?
can_ride('Zipper', 'Adult')# what will this return?
```

Rides With Age Group, Second Attempt

```
def can_ride(ride, age):
    result = '?'
    if ride == 'Cycle Jump':
        if age == 'Child':
            result = 'Yes'
        else:
            result = 'No'
    elif ride == 'Zipper':
        if age == 'Adult':
            result = 'Yes'
        else:
            result = 'No'
    else:
        result = 'Unidentified Ride Name'
    return result
```

- This is more of a "divide and conquer" approach, we first split things up by ride and then by age group
  - (could we have split by age first instead?)

Amusement Park Rides

Now assume our park is doing so well we can afford to buy two more rides.

- Our children's rides are making the most money, so we buy two more rides for children
- Child only rides:
  - "Cycle Jump"
  - "Quad Runner"
  - "Kid Wheel"
- Adult only rides:
  - "Zipper"
- Let's update the first function we wrote

```
def can_ride(ride, age):
    result = '?'
    if ride == 'Cycle Jump' and age == 'Child':
        result = 'Yes'
    elif ride == 'Cycle Jump' and age == 'Adult':
        result = 'No'
    elif ride == 'Quad Runner' and age == 'Child':
        result = 'Yes'
    elif ride == 'Quad Runner' and age == 'Adult':
        result = 'No'
    elif ride == 'Kid Wheel' and age == 'Child':
        result = 'Yes'
    elif ride == 'Kid Wheel' and age == 'Adult':
        result = 'No'
    elif ride == 'Zipper' and age == 'Adult':
        result = 'Yes'
    elif ride == 'Zipper' and age == 'Child':
        result = 'No'
    else:
        result = 'Unidentified Ride Name'
    return result
```

- We added 4 new `elif`'s, 2 for each new ride

Let's update the second function we wrote:

```
def can_ride(ride, age):
    result = '?'
    if ride == 'Cycle Jump' or ride == 'Quad Runner' or ride == 'Kid
        if age == 'Child':
            result = 'Yes'
        else:
            result = 'No'
    elif ride == 'Zipper':
        if age == 'Adult':
            result = 'Yes'
        else:
            result = 'No'
    else:
        result = 'Unidentified Ride Name'
    return result
```

- We added 2 `or`'s to our first condition, 1 for each ride
- Often the first solution a programmer comes up with is *not* the best solution.
- We strive to write code that will be easy for us to understand, easy to debug, and easy to make changes to.

Short-circuit Evaluation

Imagine I make the following offer to every person in this class:

- Perform the following math by hand (with a piece of chalk, on the chalkboard) and if the condition evaluates to True I'll do what it tells me to do:

```
if (2 + 3 == 6) and (425445 * 134523 == 57232137735):
    I'll buy you a case of (root)beer
```

- How much time are you willing to spend performing that second calculation?
  - after finding that the first operand to `and` is `False`, you know you can't win, why do the extra work?
- `Python` is smart/lazy too!
  - when evaluating a boolean expression, `Python` stops working as soon as it *knows* the answer
  - this is known as *short-circuit* evaluation

Short-circuit Evaluation

```
def is_adult(age):
    adult = False
    if age >= 18:
        adult = True
        print('Can ride adult rides.')
    else:
        print('Can NOT ride adult rides.')
    return adult
# ====================================
def main():
    height = 55 # height = 64 # what if we had this instead?
    age = 19
    if height > 60 or is_adult(age):
        print('Height =', height, ' Age =', age)
```

- The call to `is_adult` might not happen!
  - using short-circuit evaluation to do tricky things can lead to confusing code
  - it's better to write clear straightforward code
  - avoid doing things with "side-effects" in your conditional expressions

Side Effects

Side effect: when a function or an expression alters the global state of the program (leaves the program in a different state than it found it).

This includes printing things.